

İSTANBUL SAĞLIK VE TEKNOLOJİ ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ · YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ



YAŞCA

Diş Kliniği Yönetim Sistemi

Çok-Kiracılı (Multi-Tenant) SaaS Web Uygulaması · Bitirme Projesi

Yaman Halloum
220609027 · Backend & Mimari

Ali Üre
220609040 · Full-Stack & DevOps

Cihan Kurtbey
220609001 · Frontend & API

Şükrü Yeşilmen
220609037 · QA & Test

Danışman: Dr. Öğr. Üyesi Sabri Mumcu · Grup 11 · İstanbul, 2026

> Sunum Akışı

Problemten canlı ürüne — analiz, tasarım diyagramları, metrikler ve ölçekleme

- ▶ **Problem & Motivasyon**
- ▶ **Amaç, Kapsam, Çözüm**
- ▶ **Roller & RACI**
- ▶ **Katmanlı Mimari** (Şekil 3.1)
- ▶ **BPMN İş Süreçleri** (Şekil 3.2)
- ▶ **Use-Case** (Şekil 3.3)
- ▶ **Sınıf Diyagramları** (3.4–3.5)
- ▶ **Sekans Diyagramı** (Şekil 3.6)
- ▶ **ER Diyagramı** (Şekil 3.7)
- ▶ **Multi-Tenant İzolasyon**
- ▶ **Teknoloji & Karar Matrisleri**
- ▶ **Güvenlik · API/Swagger**
- ▶ **Test & Ölçekleme Metrikleri**
- ▶ **CI/CD · Yapay Zeka**
- ▶ **Uygulama Ekranları** (4.1–4.10)

1

Problem & Motivasyon

Diş kliniklerinin dijitalleşmesinin önündeki engeller



Yüksek Maliyet

Hekim/kullanıcı başına lisans ve abonelik ücretleri küçük klinikler için büyük bir bariyer.



Hantal Arayüzler

"Herkese uyan tek beden" yaklaşımı; gereksiz modüllerle dolu, karmaşık sistemler.



Güvenlik & Stabilité

KVKK/HIPAA zafiyetleri, randevu çakışmaları, donma ve veri sıfırlanma sorunları.

Literatür (Kartal, 2015 — MHRS analizi): diş hekimliğinin **değişken tedavi süreleri** jenerik sistemlerce karşılanamıyor; sabit randevu aralıkları hekimlerce "verimsiz" bulunuyor.

→ İhtiyaç: **Radikal Basitlik** odaklı, açık kaynak (FOSS), güvenli ve ekonomik bir klinik yönetim sistemi.

Amaç & Kapsam

- ▶ Klinik süreçlerini **tek güvenli platformda** dijitalleştirmek
- ▶ Modern mimari, tasarım kalıpları, **çevik (Agile)** süreçleri uçtan uca uygulamak
- ▶ Kod kalitesi, test edilebilirlik ve sürdürülebilirlik yüksek ürün
- ▶ Kullanıcılar: Yönetici, Hekim, Asistan (MVP)
- ▶ MEDULA/e-Nabız entegrasyonu kapsam dışı

Temel Modüller

- ▶ Hasta kaydı & tıbbi özgeçmiş (anamnez)
- ▶ Çakışma kontrollü **randevu planlama**
- ▶ İnteraktif **Dijital Diş Şeması** (Odontogram/FDI)
- ▶ Tedavi takibi & ödeme yönetimi
- ▶ Dashboard, klinik ayarları, işlem geçmişi

Çok-Kiracılı SaaS

FOSS

Canlı: yascadental.com

3 Kullanıcı Roller ve RACI Matrisi

RBAC mimarisi RACI matrisi referans alınarak kodlandı

Rol	Açıklama	Erişim ve Yetkiler
Yönetici (Admin)	Başhekim / klinik sahibi	Klinik ayarları, personel yönetimi, tüm verilere tam erişim, şifre sıfırlama, işlem geçmişi
Hekim (Doktor)	Sistemin ana kullanıcısı	Kendi randevuları, tedavi girişi, odontogram, dashboard
Asistan / Sekreter	İdari süreçler	Hasta kaydı, randevu oluşturma, ödeme girişi

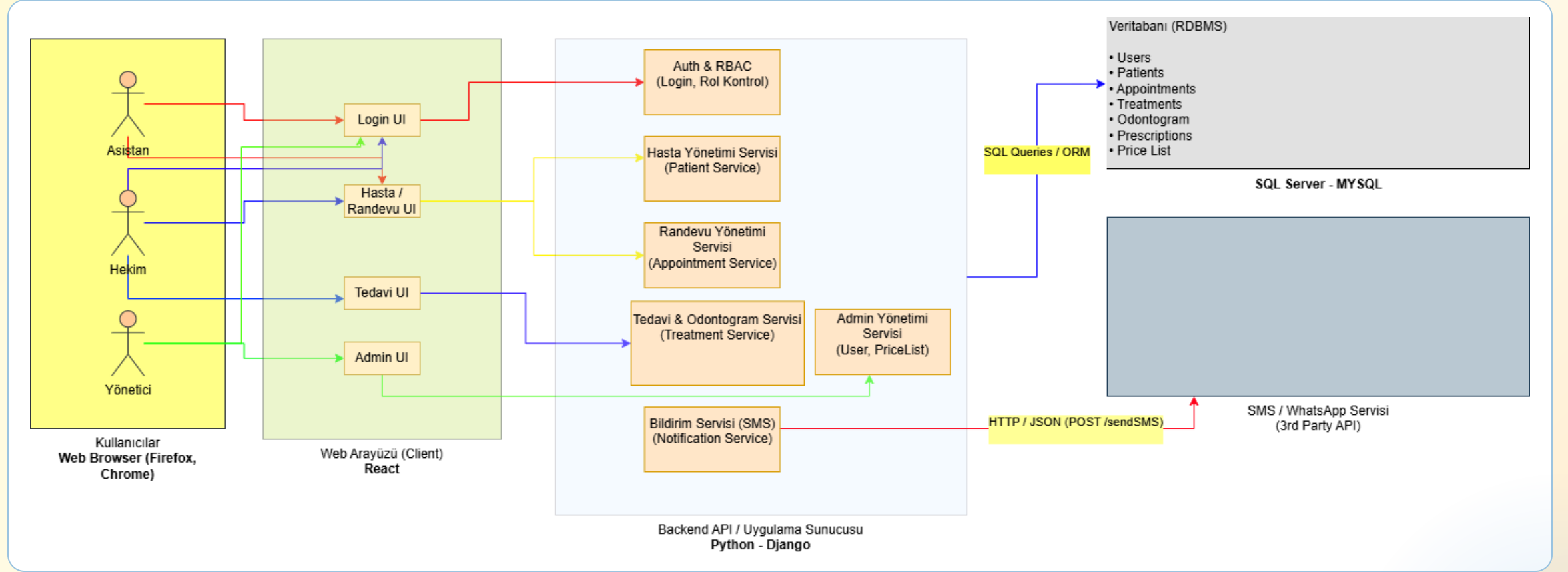
Modül / Fonksiyon	Hekim	Asistan	Yönetici
Yeni Hasta Kaydı	I	R	A
Anamnez Girişi	R	C	I
Randevu Oluşturma/İptal	I	R	A
Odontogram İşlem Kaydı	A/R	I	I
Klinik Ayarları & Fiyat	C	—	R/A

R: Sorumlu · A: Onaylayan · C: Danışılan · I: Bilgilendirilen

4

Sistem Mimarisi — Katmanlı Yapı

Decoupled + Monolitik hibrit · Boundary-Control-Entity (Şekil 3.1)



Şekil 3.1: Sistemin Katmanlı Mimari Yapısı

Boundary: React arayüzü (Login, Hasta/Randevu, Tedavi, Admin UI)

Control: Django servisleri (Auth&RBAC, Patient, Appointment, Treatment, Notification)

Entity: PostgreSQL — arayüz DB ile **asla doğrudan** konuşmaz

Neden ayrık istemci-sunucu + monolitik backend?

Decoupled (API-First)

- ▶ İstemci (React) ve sunucu (Django) **fiziksel olarak ayrı**
- ▶ JSON tabanlı REST API üzerinden haberleşme
- ▶ Modülerlik + gelecekte mobil platforma açılım
- ▶ **Monorepo:** /frontend, /backend, /docs tek depoda

Neden Monolitik Backend?

- ▶ **ACID bütünlüğü:** tedavi + ödeme tek transaction'da tutarlı
- ▶ **Düşük karmaşıklık/maliyet:** mikroservis ağ gecikmesi yok
- ▶ **Yüksek hız:** Django "batteries-included" → hızlı time-to-market

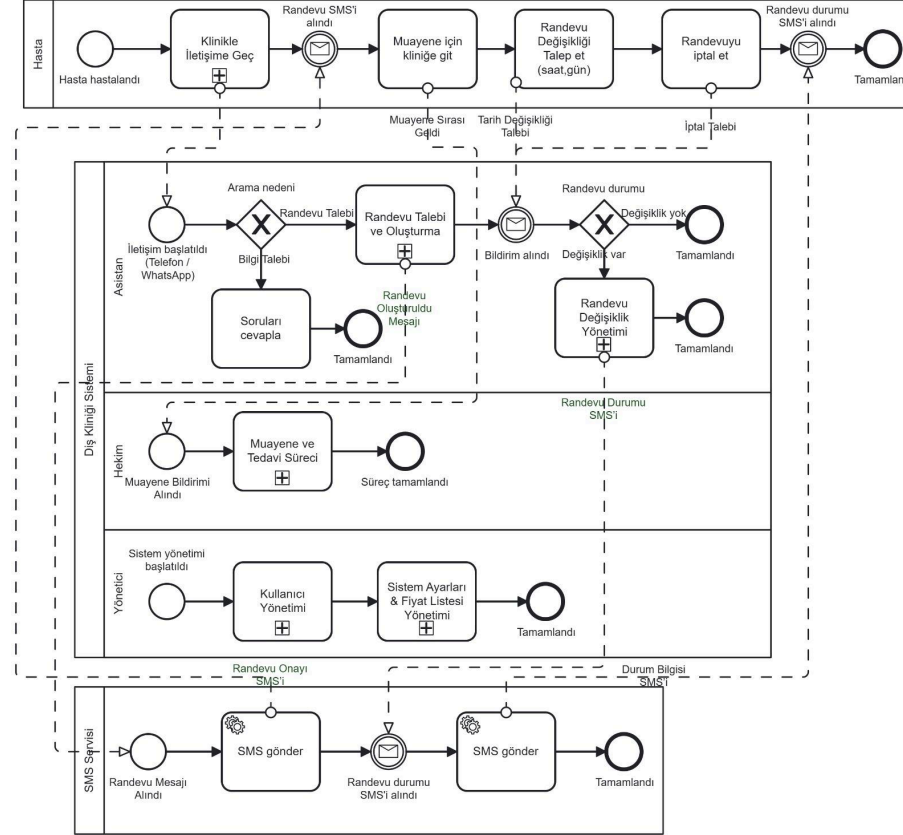
REST

JSON/HTTP

MVT/MVC

İş Süreçleri — BPMN Modeli

Klinik akışları BPMN 2.0 ile modellendi; darboğazlar otomatik kısıtlamaya dönüştü (Şekil 3.2)



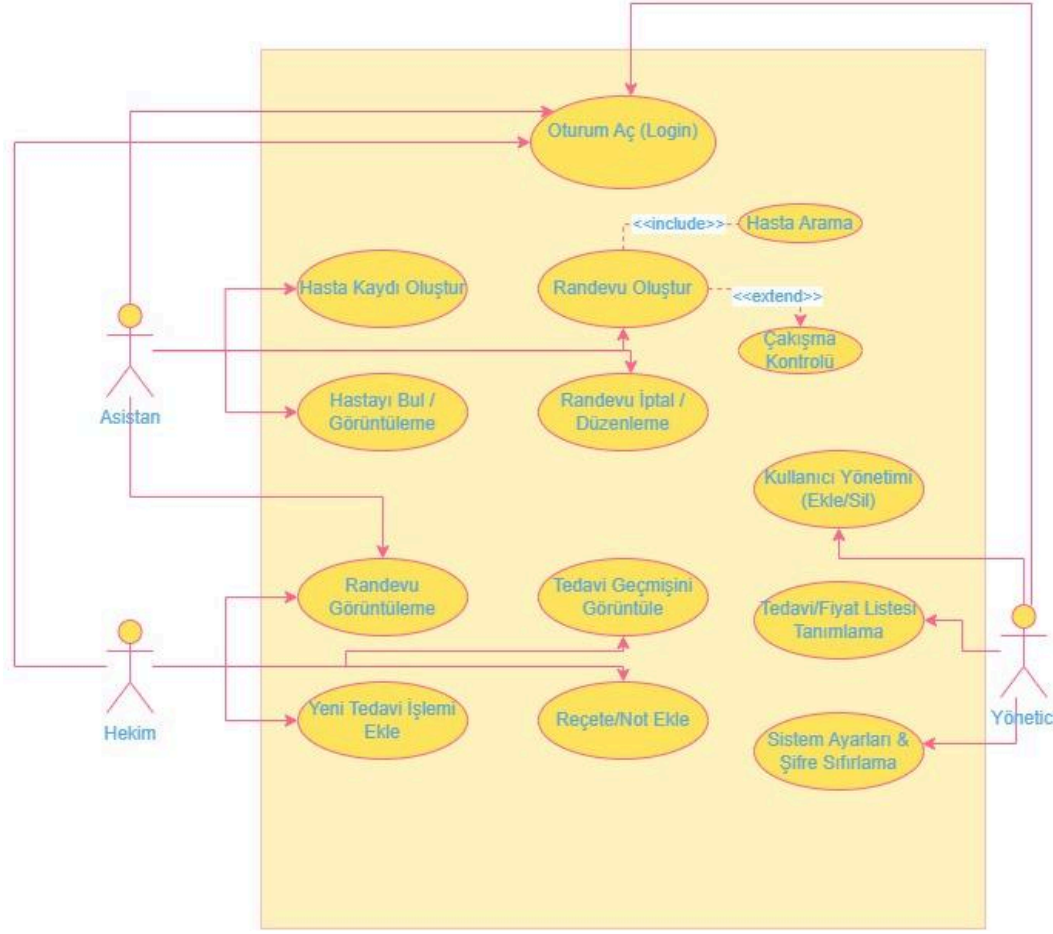
Şekil 3.2: BPMN Modeli — Hasta, Asistan, Hekim, Yönetici ve SMS Servisi havuzları

Amaç: hasta kabul–tedavi–faturalandırma akışlarındaki darboğazları kod öncesi tespit

Çıktı: randevu çakışması gibi insan hataları sistemde otomatik **constraint** oldu

Kullanım Durumları (Use-Case)

Üç aktörün sistemle etkileşim senaryoları (Şekil 3.3)



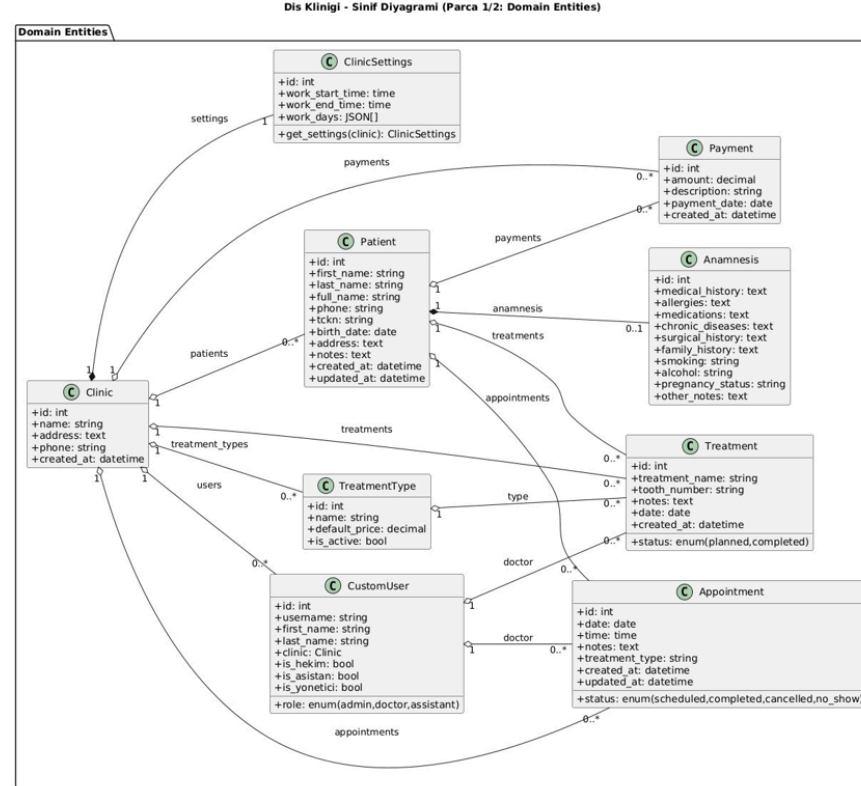
Temel Kullanım Durumları

- ▶ **Giriş yap** — tüm roller, JWT
- ▶ **Hasta kaydı** — Asistan, Yönetici
- ▶ **Randevu oluştur** — çakışma kontrollü
- ▶ **Tedavi gir** — Hekim, odontogram
- ▶ **Ödeme al** — Asistan, Yönetici
- ▶ **Klinik ayarları** — yalnız Yönetici
- ▶ **Personel yönet** — Yönetici

Şekil 3.3: Use-Case (Kullanım Durumu) Özeti

Sınıf Diyagramı — Domain Katmanı

Varlık (Entity) modelleri ve ilişkileri (Şekil 3.4)



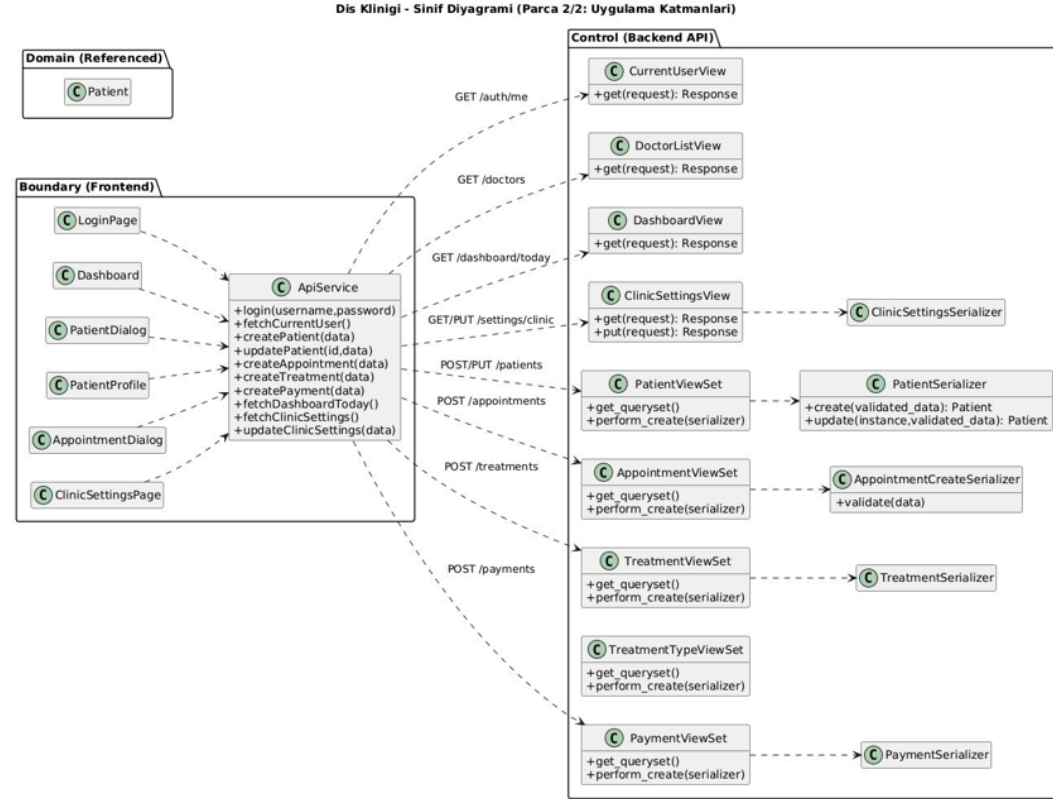
Şekil 3.4: Domain (Varlık) Katmanı Sınıf Diyagramı

Merkez: Clinic & Patient · 1:1 Anamnesis · 1:N Appointment/Treatment/Payment

Destek: CustomUser, TreatmentType, ClinicSettings · enum durumlar (scheduled/completed/cancelled)

Sınıf Diyagramı — Uygulama Katmanı

Boundary (UI) ↔ Control (api.ts / ViewSet / Serializer) (Şekil 3.5)



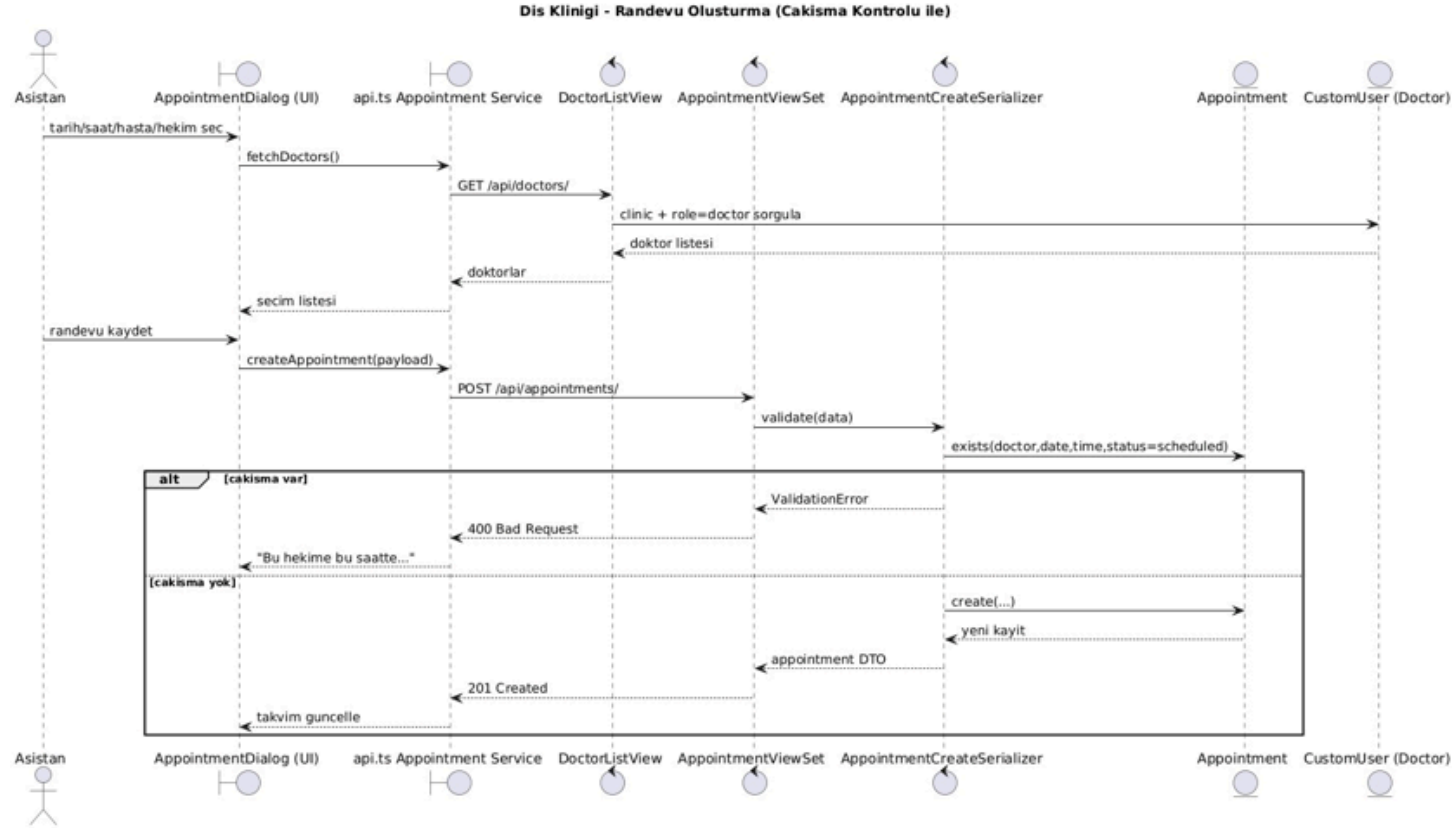
Şekil 3.5: Uygulama Katmanı (Boundary/Control) Sınıf Diyagramı

Sınıf-sekans tutarlılığı: arayüz mesajları (createAppointment, fetchDashboardToday) Control metodlarıyla birebir eşleşir

Merkezi api.ts : JWT ve X-Tenant başlıklarını her isteğe ekler

Sekans Diyagramı — Randevu & Çakışma

SD-03: Çakışma kontrollü randevu oluşturma akışı (Şekil 3.6)

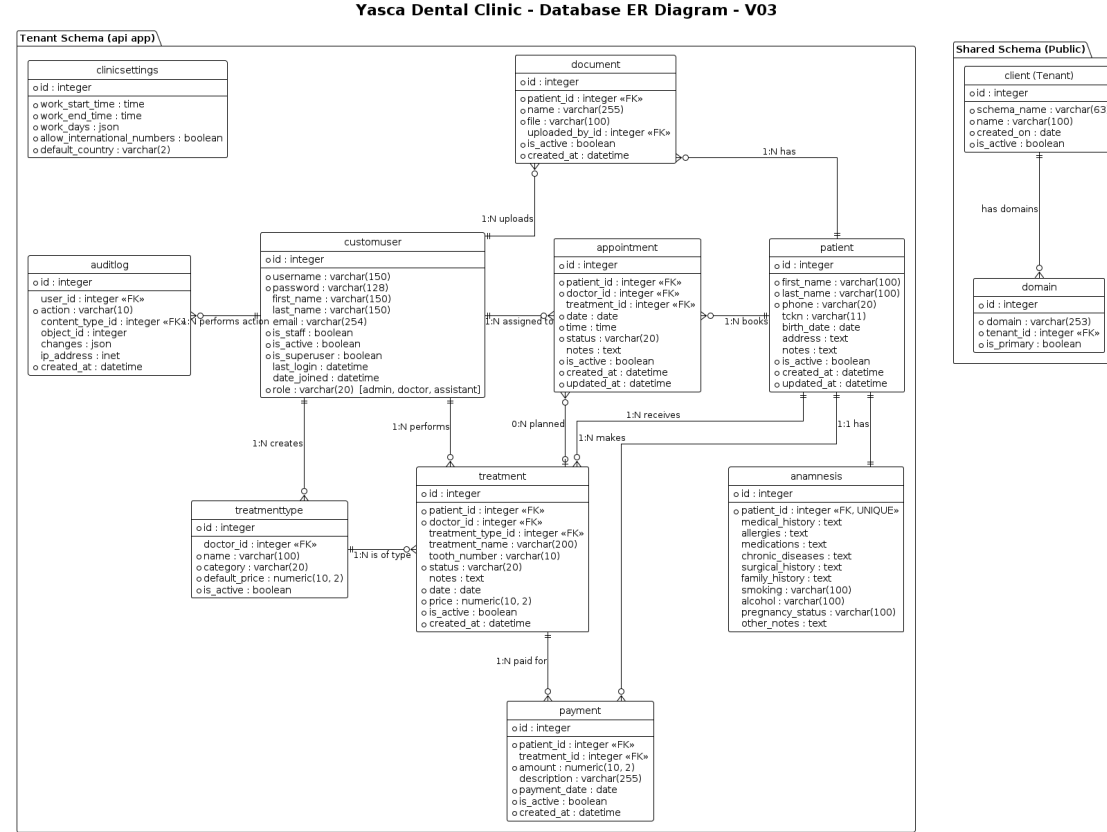


Şekil 3.6: Randevu Oluşturma ve Çakışma Kontrolü Sekans Diyagramı (SD-03)

Çakışma var → Serializer `ValidationError` → HTTP 400 "Bu hekime bu saatte..."

Çakışma yok → kayıt oluşturulur → HTTP 201 Created → takvim güncellenir

3. Normal Form · public + tenant şema ayrımı (Şekil 3.7)



Şekil 3.7: ER Diyagramı (Yasca Dental Clinic — Database ERD)

Tenant şema: Patient, Appointment, Treatment, Payment, Anamnesis, AuditLog

Shared (public) şema: Client (tenant) & Domain — kiracı yönetimi

Temel varlıklar ve ilişkileri

Varlık	Açıklama	Temel İlişkiler
Patient	Hasta kaydı (sistemin merkezi)	1:1 Anamnesis · 1:N Appointment/Treatment/Payment/Document
CustomUser	Klinik personeli	1:N Appointment, Treatment, TreatmentType
Appointment	Çakışma kontrollü randevu	N:1 Patient, Doctor · opsiyonel Treatment
Treatment	Tedavi (diş no, FDI)	N:1 Patient, Doctor, TreatmentType · 1:N Payment
Payment	Ödeme kaydı	N:1 Patient · opsiyonel Treatment
AuditLog	İşlem geçmişi (C/U/D)	N:1 User · GenericForeignKey

 Çakışma kuralı: aynı hekime, aynı tarih-saatte yalnızca bir "planlanmış" randevu — DB + serializer katmanında engellenir. Tüm tablolar 3NF.

Multi-Tenant İzolasyon

Schema-per-Tenant · PostgreSQL ile fiziksel veri izolasyonu



Kiracı Modeli

- ▶ Her klinik = kendi **izole PostgreSQL şeması**
- ▶ Ortak veri `public` ; her klinik için `tenant_klinikA` otomatik
- ▶ Olası zafiyette dahi klinikler birbirinin verisine **erişemez**

Benzetme: aynı bina (yazılım), ayrı daireler (klinik) — kimse komşunun içeri giremez.



İstek Çözümleme (Middleware)

- ▶ 1. `X-Tenant` başlığı okunur
- ▶ 2. Yedek: `Host` → domain eşlemesi
- ▶ 3. Bulunamazsa `public` şemaya düşer
- ▶ 4. `connection.set_tenant()` → bağlantı yönlendir

Frontend: canlıda `/app/:slug` , yerelde subdomain.

Katman	Teknolojiler
Ön Yüz	React 18, TypeScript, Vite, Tailwind CSS v4, Radix UI / MUI, react-router, react-hook-form, recharts, react-dnd, i18next (TR/EN)
Arka Uç	Python 3.12, Django 5.2, Django REST Framework, simplejwt, django-tenants, drf-spectacular, django-filter, django-anymail[resend], gunicorn
Veritabanı	PostgreSQL — schema-per-tenant izolasyon
Test	Vitest, Testing Library, MSW, Playwright, vitest-axe, Stryker, pytest, factory-boy + Faker, Pact
DevOps	Docker & docker-compose, GitHub Actions, Vercel (ön yüz), Render (arka uç + PostgreSQL), Codecov

Neden React?

Odontogram'ın yoğun DOM işi için Virtual DOM ile anlık güncelleme.

Neden Django?

Yerleşik güvenlik (Auth/RBAC, CSRF, SQLi) ve hız.

Neden PostgreSQL?

ACID, ilişki modelleme ve şema bazlı izolasyon.

Teknoloji Karar Matrisleri

Ağırlıklı puanlama ile gerekçelendirilmiş seçimler (5 = en iyi)

Backend	DRF	Express	Spring
Geliştirme hızı	5	4	2
Dahili güvenlik	5	2	4
Ekip yetkinliği	5	3	2
OOP uyumu	4	3	5
TOPLAM	19	12	13

Frontend	React	Vue	Angular
Topluluk	5	4	3
DOM perf.	5	4	3
Ekip yetkinliği	5	2	1
Bileşen uyumu	5	5	5
TOPLAM	20	15	12

Veritabanı	PgSQL	Mongo	SQLite
ACID	5	2	4
İlişki modeli	5	3	5
Ölçeklenebilirlik	5	5	1
Veri güvenliği	5	3	2
TOPLAM	20	13	12

Seçimler: Django REST Framework · React · PostgreSQL — her kategoride en yüksek puan.



Kimlik Doğrulama (JWT)

Durumsuz Access + Refresh token. 401'de sessiz token yenileme ile kesintisiz oturum.



Yetkilendirme (RBAC)

Admin/Hekim/Asistan rol sınırları backend'de zorlanır. Hekim yalnız kendi verisini görür.



Kiracı İzolasyonu

HeaderTenantMiddleware her istekte şemayı değiştirir; klinikler arası sızıntı mimari düzeyde engellenir.



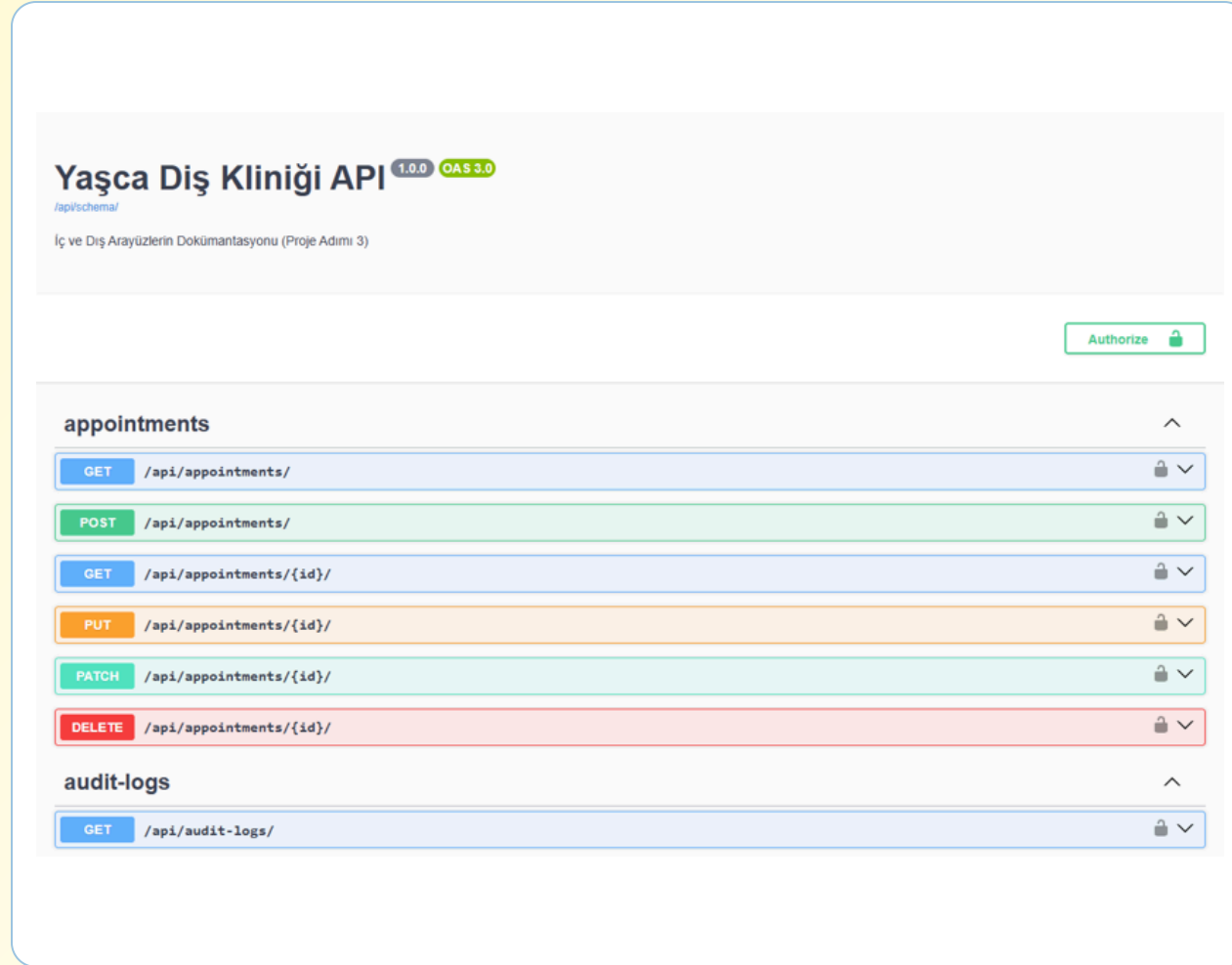
Denetim İzi (Audit Log)

Tüm CREATE/UPDATE/DELETE; kullanıcı, IP ve değişiklik detayıyla kalıcı kaydedilir.

Ek koruma: Django yerleşik **SQL Injection** & **CSRF** savunmaları · otomatik `Authorization` + `X-Tenant` başlıkları.

14 REST API & Swagger

Kaynak odaklı uç noktalar · drf-spectacular ile canlı dokümantasyon (Şekil 3.8)



Şekil 3.8: Swagger (OpenAPI) Canlı API Dokümantasyonu

Uç Nokta	Metot
/api/auth/token/	POST
/api/auth/me/	GET
/api/patients/	GET/POST
/api/appointments/	GET/POST
/api/treatments/	GET/POST
/api/payments/	GET/POST
/api/dashboard/today/	GET

CRUD eylemleri URL'e gömülmez; HTTP fiilleriyle belirtilir. Çoğul, küçük harf, öngörülebilir API sözleşmeleri.

Test Stratejisi

Test Piramidi · matrix (SQLite + PostgreSQL) koşum

E2E · Playwright — 83 (%20,6)

Backend · pytest — 169 (%41,9)

Frontend Birim · Vitest — 151 (%37,5)

Toplam **403 otomatik test** · hızlı geri bildirim (SQLite) + gerçek izolasyon (PostgreSQL).



Test Altyapısı

- ▶ **factory-boy + Faker:** izole sentetik veri
- ▶ **seed_demo_data:** çok-kiracılı demo tek komut
- ▶ **mock/monkeypatch:** dış bağımlılık taklidi
- ▶ **Stryker:** mutasyon testi (test etkinliği)
- ▶ **Pact:** sözleşme testleri
- ▶ **Lighthouse + vitest-axe:** perf & WCAG 2.1 AA

Test Metrikleri

Katman bazlı kapsam & başarı oranları (CI/CD doğrulamalı)

403

Toplam Otomatik Test

%100

Genel Başarı Oranı

%81,7

Ortalama Kod Kapsamı

AA

WCAG 2.1 Erişilebilirlik

Test Katmanı	Teknoloji	Adet	Başarı	Kapsam
Ön Yüz Birim	Vitest & Testing Library	151	%100	%82,4
Arka Uç Birim	pytest & Django TestCase	169	%100	%81,0
Uçtan Uca (E2E)	Playwright	83	%100	—
Erişilebilirlik (A11y)	vitest-axe	Kritik arayüzler	WCAG AA	%100
TOPLAM	CI/CD Pipeline	403	%100	%81,7

Backend asgari kapsam geçidi **%78** zorunlu — eşiği geçemeyen kod ana dala birleştirilemez (Codecov).

Yük Testi Tasarımı

- ▶ Araç: **Locust** (headless, CI manuel tetik)
- ▶ **50 eşzamanlı** sanal kullanıcı · spawn 5/sn · 1 dk
- ▶ Her kullanıcı **rastgele tenant** → **X-Tenant** ile gerçek şema geçiş maliyeti ölçülür
- ▶ Gerçekçi düşünme süresi: istekler arası 1–3 sn
- ▶ PostgreSQL modunda gerçek izolasyon altında koşulur

"Sistem 1 kullanıcıyla çalışır; ya 500 klinik aynı anda istek atarsa?" sorusunun yanıtı.

Okuma-Ağırlıklı Trafik Profili

Senaryo (Endpoint)	Ağırlık
GET /dashboard/today/	5×
GET /patients/	4×
GET /appointments/	3×
GET /treatments/	2×
GET /patients/?search=	1×

Çıktı: p95 yanıt süresi, RPS ve hata oranı CSV olarak raporlanır; **p95 eşiği** fail kriteridir.

18

Mimari Ölçeklenebilirlik

Yatay büyümeye hazır, durumsuz ve izole tasarım



Durumsuz API

JWT ile stateful oturum yok → gunicorn worker'ları **yatay ölçeklenir**.



Schema-per-Tenant

Yeni klinik = yeni şema; büyüme veri karışması riski olmadan eklenir.



Yönetilen Bulut

Vercel (CDN) + Render (managed PostgreSQL) → altyapı ölçekleme devredildi.

50

Yük testi eşzamanlı kullanıcı

2+

Paralel test edilen kiracı

p95

İzlenen ana ölçekleme metriği

Strateji: **API-first + monolit** ile başla, gerektiğinde worker çoğalt; veri katmanı şema bazında bölümlenmiş kalır.

19

Kod Tabanı & Efor Metrikleri

Statik analiz ile ölçülen mühendislik hacmi

41.473

Satır Kod (LOC)

282

Dosya

~1.602

Fonksiyon / Metot

778

Adam-Saat Efor

119 commit · 4 kişilik full-stack ekip · iteratif-artımlı Agile.

Uzanti	Katman	Dosya	LOC
.tsx	React UI	104	15.739
.json	Yapılandırma / i18n	30	10.218
.py	Django Backend	69	6.741
.md	Dokümantasyon	28	4.542
.ts	Tipler / Servisler	32	2.703
TOPLAM	Monorepo	282	41.473

CI/CD & Kalite Geçitleri

GitHub Actions ile otomatik kalite — "yeşil olmadan merge yok"

İş Akışları

- ▶ `ci.yml` — backend (SQLite+PgSQL), frontend (tsc+eslint+vitest), a11y
- ▶ `e2e-tests.yml` — Playwright
- ▶ `contract-tests.yml` — Pact
- ▶ `mutation.yml` — Stryker
- ▶ `lighthouse.yml` — perf / a11y
- ▶ `load-test.yml` — Locust yük testi

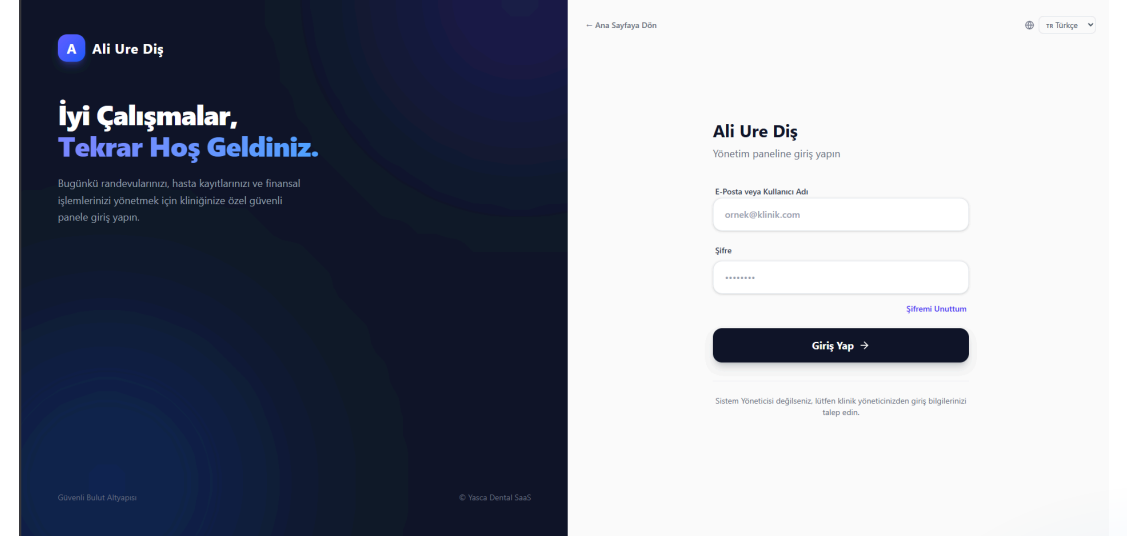
Kalite Geçitleri

- ▶ **Branch protection:** main'e push yasak, PR zorunlu
- ▶ **CODEOWNERS:** en az 1 onay (code review)
- ▶ **Pre-commit:** husky + lint-staged (zero-warning)
- ▶ **Conventional Commits** + squash & merge
- ▶ **Coverage %78** altı merge engellenir
- ▶ **Deploy:** testler yeşilse Vercel + Render'a otomatik

SaaS Landing ve JWT tabanlı klinik girişi (Şekil 4.1–4.2)

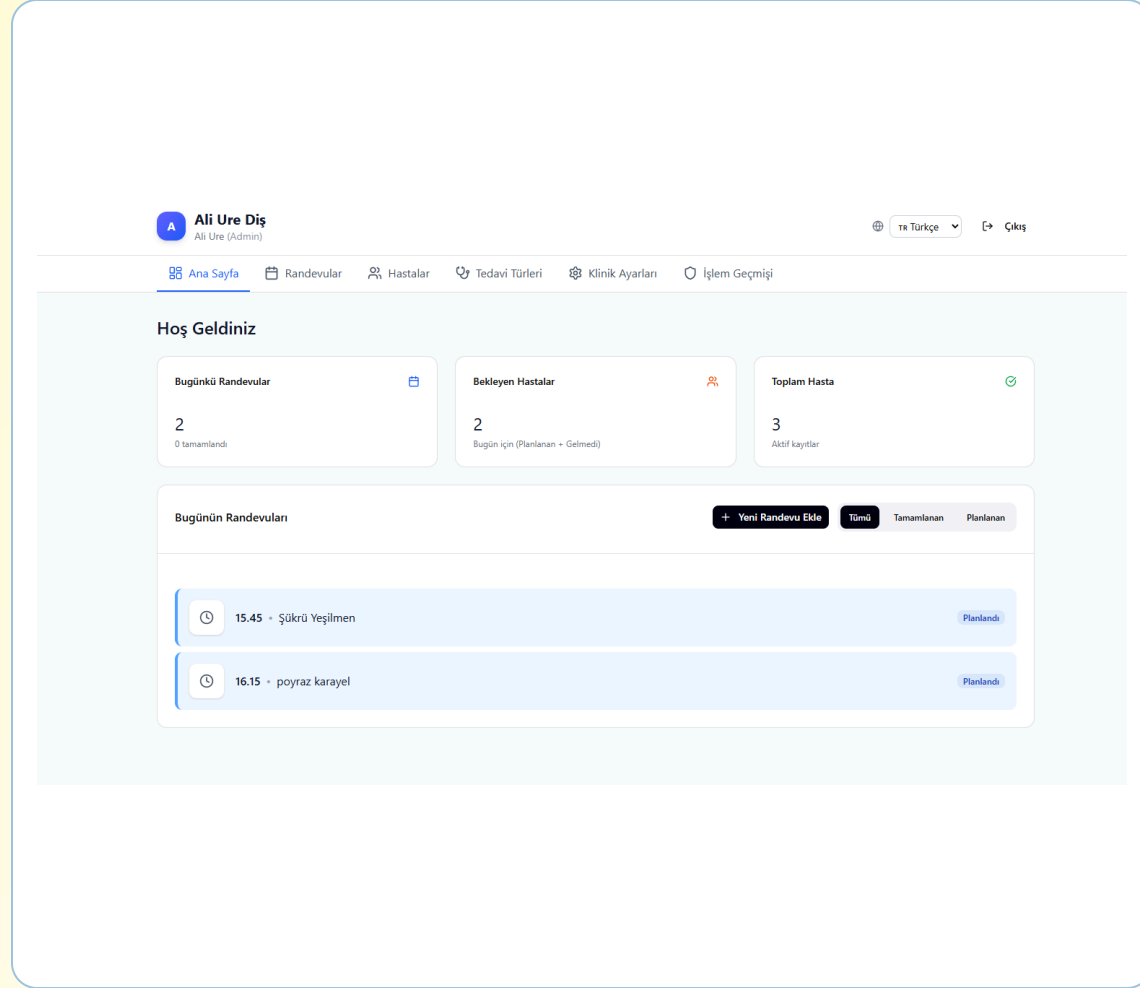


Şekil 4.1: SaaS Tanıtım Sayfası

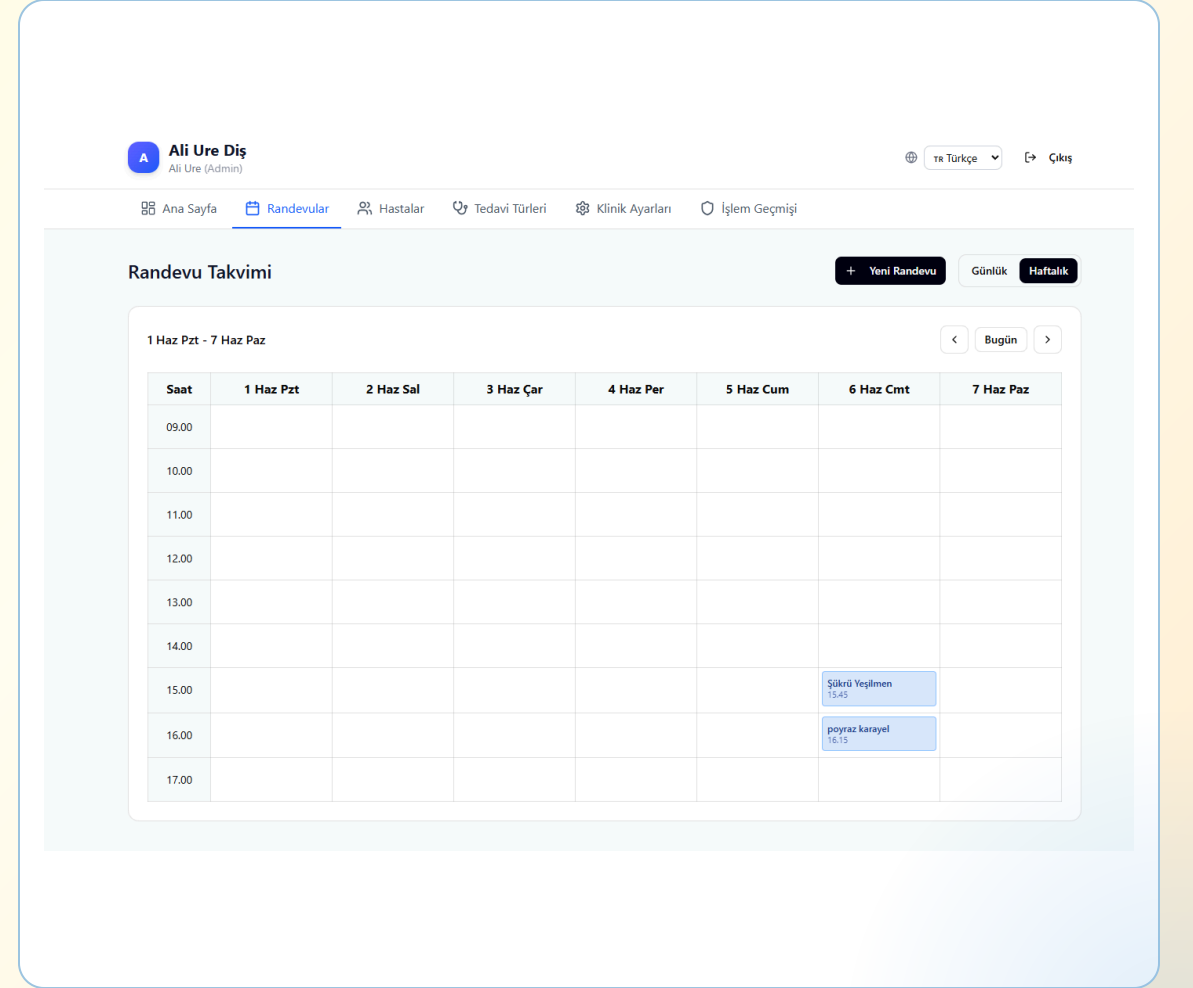


Şekil 4.2: Klinik Giriş Sayfası

Minimalist dashboard ve çıkışma kontrollü takvim (Şekil 4.3–4.4)

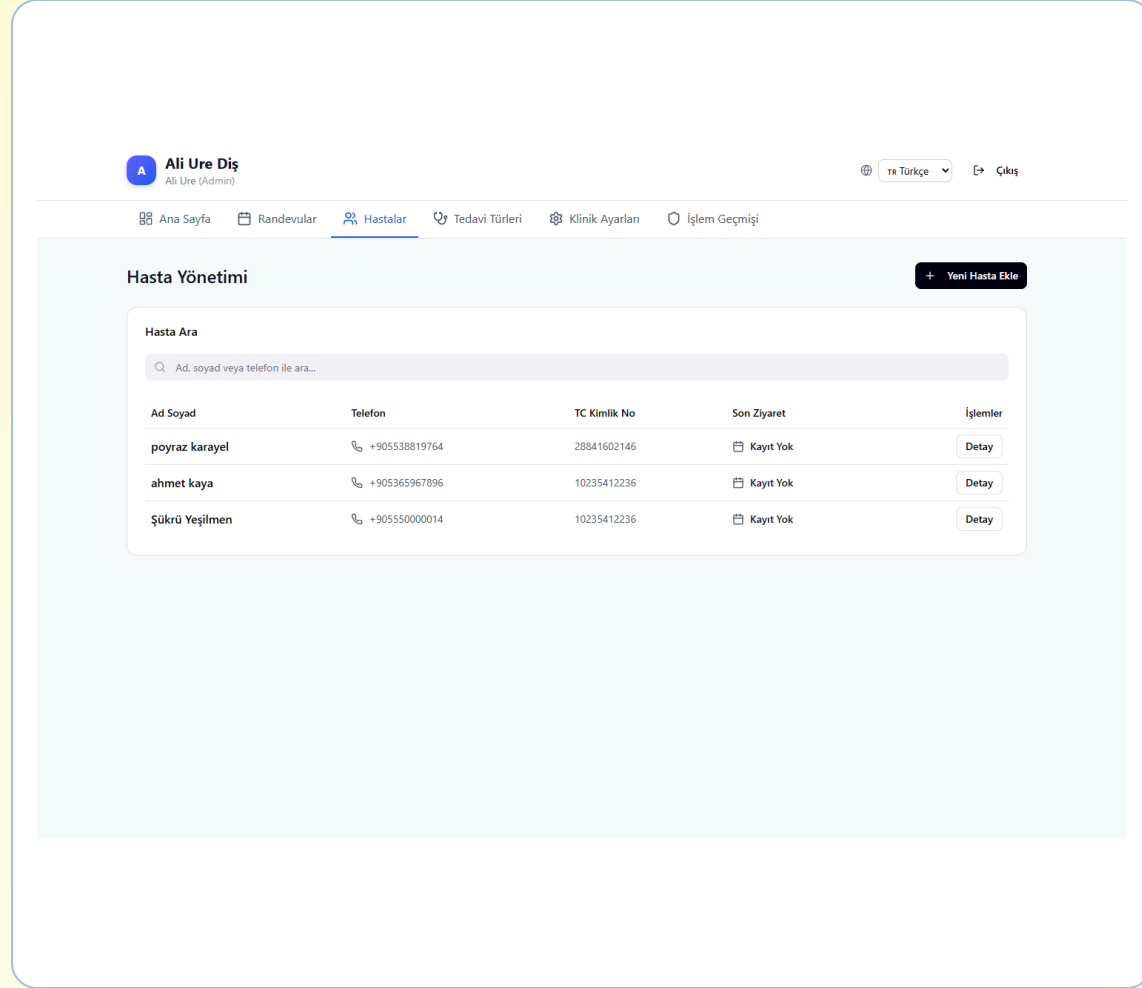


Şekil 4.3: Dashboard

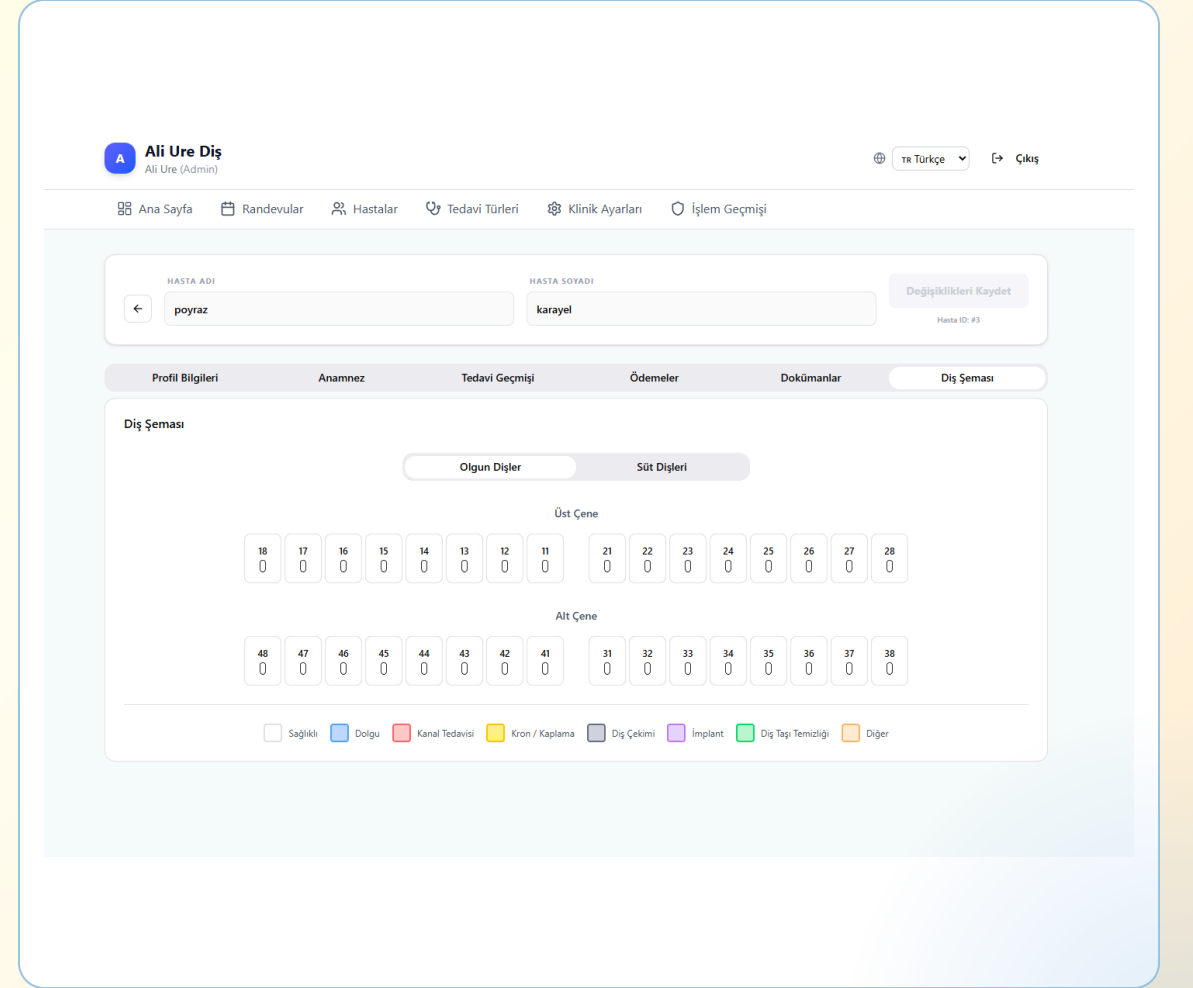


Şekil 4.4: Randevu Takvimi

Hasta yönetimi ve projenin kalbi: interaktif Dijital Diş Şeması (Şekil 4.5–4.6)



Şekil 4.5: Hasta Listesi



Şekil 4.6: Diş Şeması (FDI, renk kodlu)

Uygulama Ekranları — Yönetim & Geçmiş

Tedavi türleri, klinik ayarları ve denetim izi (Şekil 4.7–4.10)

Şekil 4.7: Tedavi Geçmişi

Şekil 4.8: Tedavi Türleri

Şekil 4.9: Klinik Ayarları

Şekil 4.10: İşlem Geçmişi (Audit Log)

Yapay Zeka Kullanım Pratikleri

GenAI bir "güç çarpanı" — otomatik pilot değil · human-in-the-loop

Aşama	Yapay Zeka Kullanımı	İnsan Denetimi
Tasarım / Analiz	Gereksinim netleştirme, UML & veri modeli taslakları	Kararların ekipçe onayı
Kodlama	Model/serializer/viewset & React iskeletleri, boilerplate	Kod gözden geçirme, refactoring
Test	Pozitif/negatif senaryolar, veri fabrikaları	Çalıştırıp doğrulama
Hata Ayıklama	Hata yorumlama, kök neden & çözüm önerileri	Uygulama ve test etme
Dokümantasyon	README/ADR/docstring taslakları	Doğrulama & düzenleme

Güvenlik ilkesi: şifre, TCKN gibi **hassas veriler asla** prompt'lara gömülmedi; üretilen hiçbir kod test ve insan denetiminden geçmeden merge edilmedi.

✓ Başarılar / ⚠ Dersler

- ▶ 403 testlik altyapı + CI/CD ile güvenli merge
- ▶ schema-per-tenant ile yapısal veri güvenliği
- ▶ Merkezî `api.ts` (BCE) ile sade entegrasyon
- ▶ ⚠ Birim test oranı %37,5 — hedef %70'in altında
- ▶ ⚠ django-tenants karmaşıklığı → özel middleware

💡 Öneriler

- ▶ İş mantığını ayrı **Service Layer**'a taşımak
- ▶ Birim test oranını artırmak · RFC 7807 hata yönetimi

📅 Sürüm Yol Haritası

Sürüm	Özellikler	Tarih
v1.1	SMS/WhatsApp, takvimde drag & drop	2026 Q3
v1.2	Raporlama modülü, PDF çıktı	2026 Q4
v2.0	Mobil (React Native), çok dilli	2027 Q1
v2.1	Röntgen entegrasyonu, analitik	2027 Q2

v1.0 (MVP) **canlı yayında**: yascadental.com



Teşekkür Ederiz

Yaşca Diş Kliniği Yönetim Sistemi — açık kaynaklı, çok-kiracılı, test edilmiş ve canlı bir SaaS mühendislik ürünü.

Yaman Halloum

Ali Üre

Cihan Kurtbey

Şükrü Yeşilmen

Sorularınız için hazırız · Grup 11 · Danışman: Dr. Öğr. Üyesi Sabri Mumcu